

프로젝트 결과 보고서

프로젝트명 : 학생 생활패턴 기반 최종 성적 예측 및 학업 위험군 분류

01 프로젝트 개요와 모델 설계

한 줄 소개

- UCI STUDENT PERFORMANCE DATASET을 활용해 회귀 모델로 최종 성적을 예측하고, 분류 모델로 위험/보통/우수 그룹을 판정한 프로젝트입니다.

프로젝트 목표

- 목표: 학생 생활패턴과 과거 성적 데이터를 활용해 최종 성적 G3를 예측하고, 학업 위험군을 분류하는 프로젝트를 빠르게 보여준다.

문제 정의

- 최종 성적이 나온 뒤에는 학업 위험군 대응이 늦다.
- 생활패턴, 결석, 가정환경, 과거 성적을 활용하면 사전 예측 가능성이 확인할 수 있다.
- 단순 성적 예측이 아니라 입력 변수 구성에 따른 성능 차이를 비교했다.

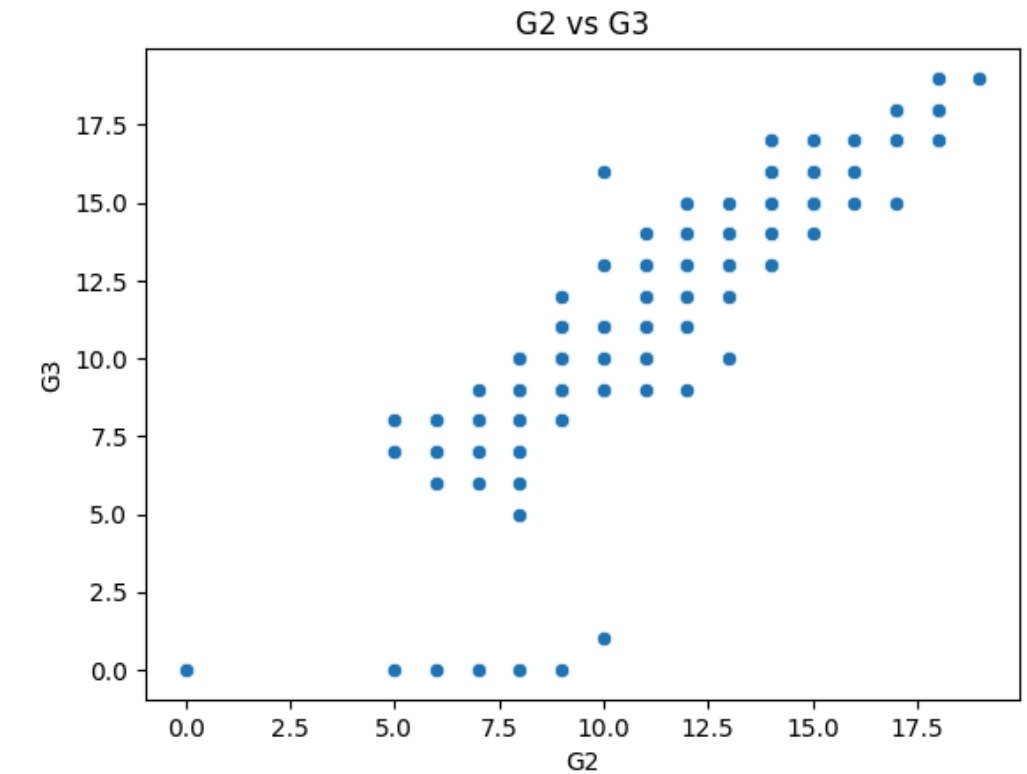
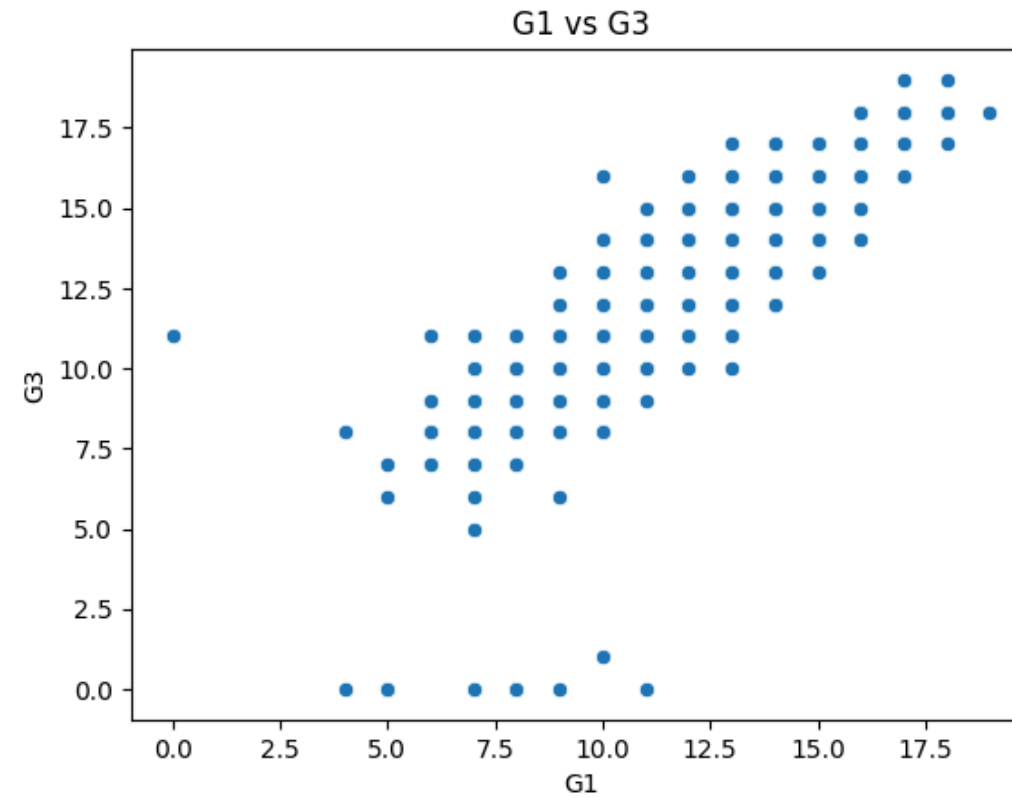
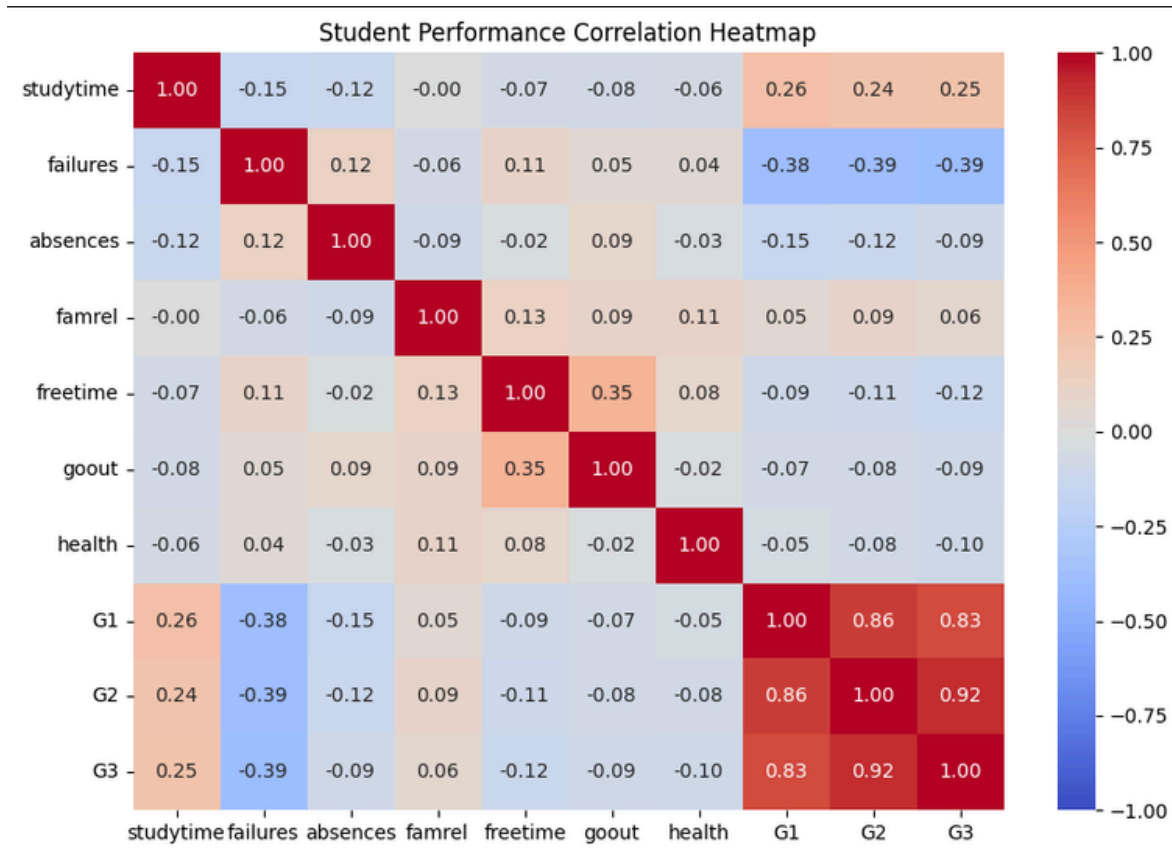
데이터와 모델

- 데이터: UCI STUDENT PERFORMANCE DATASET
- 규모: 649행, 33개 컬럼
- TARGET 1: G3 최종 성적 점수
- TARGET 2: 위험/보통/우수 학업 수준 그룹
- MODEL A: 생활패턴, 학교생활, 가정환경, 결석 정보
- MODEL B: MODEL A + G1, G2 과거 성적

분석 흐름

데이터 확인 -> EDA -> 인코딩
-> TRAIN/TEST SPLIT
-> 회귀/분류 모델 학습
-> 성능 비교
-> STREAMLIT 예측 화면

01 프로젝트 개요와 모델 설계



#모델 A/B용 변수 나누기, 입력값과 정답값을 나눌 준비하기

```
y_reg = df["G3"]  
y_clf = df["grade_group"]
```

Python

개념

- 회귀: 최종 성적 점수 예측
- 분류: 위험/보통/우수 그룹 판정
- 평가지표: RMSE, R2, ACCURACY, F1-SCORE

02 결과, 화면

핵심 결과

- 회귀 최고 모델: LINEARREGRESSION_B
- RMSE: 1.215
- R2: 0.849
- 분류 최고 모델: LOGISTICREGRESSION_B
- ACCURACY: 0.869
- F1-SCORE: 0.869

구분	최고 모델	핵심 지표	해석
회귀	LinearRegression_B	RMSE 1.215 / R2 0.849	과거 성적 포함 시 예측력 상승
분류	LogisticRegression_B	F1 0.869	위험군 분류 성능 가장 우수

포르투갈 학생 생활패턴 기반 성적 예측

Student Performance Dataset 기반 머신러닝 개인 프로젝트

프로젝트 설명

포르투갈 중등학생의 생활패턴, 결석 정보와 과거 성적을 활용해 최종 성적 G3와 학업 수준을 예측합니다.

학생 수

평균 G3

최다 그룹

649명

11.91점

보통

입력값 요약

나이

수정

19세

공부 시간

수정

2단계

낙제

수정

0회

결석

수정

2일

건강

수정

4단계

가족

수정

4단계

자유

수정

3단계

외출

수정

3단계

인터넷

수정

yes

진학

수정

yes

G1

수정

16점

G2

수정

19점

예측 결과 모델 비교 데이터 정보

현재 입력값 기준 예측 결과

최종 성적 G3 19.2점 / 20점

우수 그룹

현재 입력값에서는 우수 그룹으로 분류됩니다. 과거 성적과 생활패턴 기준에서 긍정적인 예측 결과입니다.

예측 G3

학업 수준

사용 모델

19.2

우수

Model B

↑ +7.3 vs 평균

한 줄 해석: 현재 조건에서는 G3가 19.2점으로 예측되며, 학업 수준은 우수 그룹입니다.

모델 성능 요약

회귀 최고 모델

LinearRegression_B

RMSE 1.215 / R2 0.849

분류 최고 모델

LogisticRegression_B

Accuracy 0.869 / F1 0.869

03 트러블 슈팅, 소감

트러블 슈팅

```
def default_regression_result() -> pd.DataFrame:
    return pd.DataFrame(
        {
            "Model": ["LinearRegression_A", "LinearRegression_B",
                     "KNNRegression_A", "KNNRegression_B"],
            "MAE": [2.156382, 0.765060, 2.275385, 1.556923],
            "MSE": [8.189784, 1.475909, 9.066154, 4.646769],
            "RMSE": [2.861780, 1.214870, 3.011005, 2.155637],
            "R2": [0.160170, 0.848651, 0.070302, 0.523492],
        }
    )
```

1. 범주형 데이터 처리

문제: SCHOOL, SEX, ADDRESS 등 문자형 컬럼은 모델이 바로 학습할 수 없음
해결: 인코딩으로 숫자형 데이터로 변환

```
def default_classification_result() -> pd.DataFrame:
    return pd.DataFrame(
        {
            "Model": [
                "LogisticRegression_A",
                "LogisticRegression_B",
                "KNNClassifier_A",
                "KNNClassifier_B",
                "SVM_A",
                "SVM_B",
            ],
            "Accuracy": [0.669231, 0.869231, 0.584615, 0.715385, 0.661538, 0.823077],
            "Precision": [0.643905, 0.869617, 0.560769, 0.740321, 0.649559, 0.849212],
            "Recall": [0.669231, 0.869231, 0.584615, 0.715385, 0.661538, 0.823077],
            "F1": [0.636895, 0.869091, 0.545359, 0.686088, 0.554978, 0.799388],
        }
    )
```

2. 모델별 전처리 차이

문제: KNN, SVM은 변수 스케일에 민감함
해결: 스케일링을 적용해 모델 비교 기준을 맞춤

03 트러블 슈팅, 소감

트러블 슈팅

3. 변수 구성에 따른 성능 차이

문제: 생활패턴만으로는 G3 설명력이 낮음

해결: MODEL A/B를 분리해 G1, G2가 성능에 미치는 영향을 비교

소감

이 프로젝트를 통해 회귀와 분류 문제를 같은 데이터셋에서 비교하고, 입력 변수 구성과 전처리 방식이 모델 성능에 미치는 영향을 확인했습니다.

```
@st.cache_resource
def train_models(df: pd.DataFrame):
    x = df.drop(columns=["G3", "grade_group"])
    y_reg = df["G3"]
    y_clf = df["grade_group"]

    numeric_cols = x.select_dtypes(include=["int64", "float64"]).columns.tolist()
    categorical_cols = x.select_dtypes(include=["object"]).columns.tolist()

    preprocessor = ColumnTransformer(
        transformers=[
            ("num", StandardScaler(), numeric_cols),
            ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols),
        ]
    )

    regression_model = Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            ("model", LinearRegression()),
        ]
    )

    classification_model = Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            ("model", LogisticRegression(max_iter=1000)),
        ]
    )

    regression_model.fit(x, y_reg)
    classification_model.fit(x, y_clf)
    return regression_model, classification_model, x
```

감사합니다

THANK YOU!